W6.A4 Data Flow Assignment

Daniele Riva 902275

1 Type of analysis

For this assignment, **Avail variables analysis** (forward, all-paths) has been used. A read of x at n is safe only if x has been defined on *every* path from entry to n. This is the Forward $\cap$ All-paths section of the classification of data flow analyses.

2 gen and kill

The Avail is instantiated on *variables*, with gen and kill defined for each node n as:

$$\begin{aligned} \text{gen}(n) &= \{\, x \mid x \in \text{def}(n) \,\} \\ \text{kill}(n) &= \varnothing \quad \text{(once defined, a variable stays defined and cannot be killed} \end{aligned}$$

Boundary condition:

$$\text{Avail(entry)} = \{\text{formal parameters}\} = \{\textit{values}, \textit{cutoff}\}$$

Non-entry nodes are initialized to the universal set U of program variables, required by $\cap$ to reach a safe fixed point.

$$\text{Avail}(n) = \bigcap_{m \in \text{pred}(n)} \text{AvailOut}(m), \qquad \text{AvailOut}(n) = (\text{Avail}(n) \setminus \text{kill}(n)) \cup \text{gen}(n).$$

3. How the analysis detects uninitialized reads

$\text{use}(n) \setminus \text{Avail}(n) \neq \varnothing$ - some variable is read at n although at least one path from entry to n leaves it undefined.

$x \in \text{use}(n) \setminus \text{Avail}(n)$ if and only if x is used at n, while x is not defined on *all* paths reaching n.

4. kill and gen sets

Applying Avail, at line 33 the read *sum* is not in Avail(33). For this reason, line 33 reads a potentially uninitialized variable. Non-trivial predecessors: pred(17) = $\{13_F, 14, 26\}$, pred(33) = $\{29_F, 30\}$. For brevity we write sets in the order V, C, I, N, P, S, a standing for *values*, *cutoff*, *idx*, *count*, *processed*, *sum*, *val*.

Line	**gen**	**kill**	**Avail**	**AvailOut**	**Comments**
entry	V, C	–	∅	V, C	parameters initialized by caller
5	I	–	V, C	V, C, I	
6	N	–	V, C, I	V, C, I, N	uses V (∈ Avail)
7	P	–	V, C, I, N	V, C, I, N, P	
9	–	–	V, C, I, N, P	V, C, I, N, P	uses C
10	–	–	V, C, I, N, P	V, C, I, N, P	exit
13	–	–	V, C, I, N, P	V, C, I, N, P	uses N
14	S	–	V, C, I, N, P	V, C, I, N, P, S	
17	–	–	V, C, I, N, P	V, C, I, N, P	∩ drops S (and a)
18	a	–	V, C, I, N, P	V, C, I, N, P, a	uses V, I
20	–	–	V, C, I, N, P, a	V, C, I, N, P, a	uses a, C
21	P	–	V, C, I, N, P, a	V, C, I, N, P, a	uses P
23	P	–	V, C, I, N, P, a	V, C, I, N, P, a	uses P
26	I	–	V, C, I, N, P, a	V, C, I, N, P, a	uses I
29	–	–	V, C, I, N, P	V, C, I, N, P	uses P, N
30	P	–	V, C, I, N, P	V, C, I, N, P	uses N
33	–	–	**V, C, I, N, P**	V, C, I, N, P	**uses S; S ∉ Avail**

Node-by-node derivation:

entry: gen={V,C}, kill={} → Avail(entry)=∅, AvailOut(entry)={V,C}
5: gen={I}, kill={} → Avail(5)={V,C}, AvailOut(5)={V,C,I}
6: gen={N}, kill={} → Avail(6)={V,C,I}, AvailOut(6)={V,C,I,N}
7: gen={P}, kill={} → Avail(7)={V,C,I,N}, AvailOut(7)={V,C,I,N,P}
9: gen={}, kill={} → Avail(9)=AvailOut(9)={V,C,I,N,P}
10: gen={}, kill={} → Avail(10)=AvailOut(10)={V,C,I,N,P} # exit
13: gen={}, kill={} → Avail(13)=AvailOut(13)={V,C,I,N,P}
14: gen={S}, kill={} → Avail(14)={V,C,I,N,P}, AvailOut(14)={V,C,I,N,P,S}
17: gen={}, kill={} → Avail(17)=AvailOut(13_F)∩AvailOut(14)∩AvailOut(26)={V,C,I,N,P};
AvailOut(17)={V,C,I,N,P}
18: gen={a}, kill={} → Avail(18)={V,C,I,N,P}, AvailOut(18)={V,C,I,N,P,a}
20: gen={}, kill={} → Avail(20)=AvailOut(20)={V,C,I,N,P,a}
21: gen={P}, kill={} → Avail(21)=AvailOut(21)={V,C,I,N,P,a}
23: gen={P}, kill={} → Avail(23)=AvailOut(23)={V,C,I,N,P,a}
26: gen={I}, kill={} → Avail(26)=AvailOut(26)={V,C,I,N,P,a}
29: gen={}, kill={} → Avail(29)=AvailOut(29)={V,C,I,N,P}
30: gen={P}, kill={} → Avail(30)=AvailOut(30)={V,C,I,N,P}
33: gen={}, kill={} → Avail(33)=AvailOut(29_F)∩AvailOut(30)=**{V,C,I,N,P}**;
uses {S,P}; **S ∉ Avail(33)** ⇒ error.

5. Faulty statement

Line 33: "`return sum + processed;`" reads *sum*, but $sum \notin \text{Avail}(33)$. The triggering path is

$$\text{entry} \to 5 \to 6 \to 7 \to 9_F \to 13_F \to 17_F \to 29 \to 33$$

Control flow graph